

Verificação formal de um software de controle de semáforos em tempo real

1st Igor Becker Boos

PPGESE

UFSC

Joinville, Brasil

igor.beckerboos@gmail.com

2nd Jean Carlos Dapper

PPGESE

UFSC

Joinville, Brasil

jeandapper.carlos@gmail.com

Resumo—Este trabalho apresenta o desenvolvimento e a verificação de um sistema embarcado para controle de semáforos em tempo real, implementado em linguagem C por meio de uma máquina de estados temporizada. O sistema realiza o gerenciamento do fluxo de veículos em duas vias, controle de travessia de pedestres e modo de emergência/manutenção. Para validação do controlador, foram utilizados testes unitários com a biblioteca Unity e verificação formal utilizando a ferramenta UPPAAL. Os resultados demonstraram que todos os testes unitários foram aprovados, indicando conformidade funcional da implementação. Entretanto, a verificação formal identificou uma violação de fairness relacionada ao atendimento de pedestres, evidenciando um cenário em que a Rua B pode permanecer indefinidamente sem liberação de passagem. Os resultados mostram que testes unitários e verificação formal são técnicas complementares para aumento da confiabilidade em sistemas embarcados críticos.

Palavras-chave—Verificação formal, sistemas embarcados, testes unitários, UPPAAL, testes de software, sistemas em tempo real.

I. INTRODUÇÃO

O crescimento da mobilidade urbana e o aumento do fluxo de veículos tornam o controle de tráfego um problema cada vez mais relevante nas cidades modernas. Interseções semaforizadas desempenham papel fundamental na organização do trânsito, sendo responsáveis por garantir segurança, fluidez e organização tanto para veículos quanto para pedestres. Nesse contexto, sistemas embarcados aplicados ao controle semafórico exigem elevada confiabilidade, comportamento determinístico e mecanismos que assegurem o correto funcionamento em diferentes condições operacionais.

Além da implementação do controlador, a validação de sistemas críticos em tempo real tem recebido destaque nos últimos anos, principalmente devido à necessidade de garantir segurança funcional, previsibilidade temporal e redução de falhas durante a operação. Estudos recentes demonstram que técnicas de verificação e testes automatizados contribuem significativamente para o aumento da confiabilidade e robustez de sistemas embarcados críticos.

Diante desse cenário, este trabalho propõe realizar a verificação e testes de um sistema embarcado para controle de um cruzamento urbano composto por dois semáforos veiculares e um sistema de travessia de pedestres. O software é desenvolvido em linguagem C, e os testes são feitos utilizando testes unitários com Unity e verificação formal por meio da ferramenta UPPAAL, buscando garantir segurança, confiabilidade e correteude temporal do sistema.

II. ESPECIFICAÇÃO DO SISTEMA

O sistema consiste em um controlador embarcado responsável pelo gerenciamento de um cruzamento entre duas ruas de mão única, denominadas Rua A e Rua B, conforme Figura 1. O cruzamento utiliza dois semáforos principais, um para cada rua, com o objetivo de coordenar o fluxo de veículos de forma segura e eficiente, garantindo que apenas uma via possua permissão de tráfego por vez. Para isso, o sistema opera por meio de uma máquina de estados temporizada, alternando automaticamente os sinais entre verde, amarelo e vermelho, incluindo intervalos de segurança que evitam conflitos simultâneos entre os fluxos de veículos.

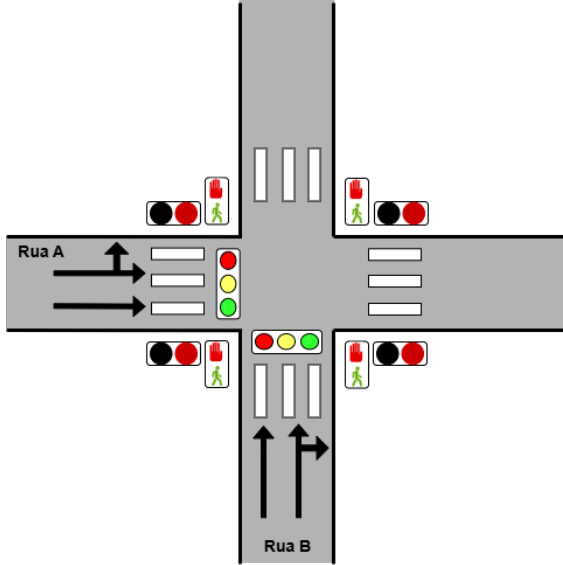


Figura 1: Ilustração do sistema de semáforos

Além do controle veicular, o sistema também gerencia a travessia de pedestres por meio de botões de solicitação instalados nas extremidades do cruzamento. Quando acionado, o pedido é registrado e atendido após a conclusão segura do ciclo atual dos veículos, momento em que ambos os semáforos passam para vermelho, liberando a travessia durante um intervalo pré-definido. O sistema também possui um modo especial de emergência/manutenção, ativado por um botão oculto acessível apenas a operadores autorizados. Nesse modo, o funcionamento automático é interrompido, ambos os semáforos permanecem em vermelho e nenhuma transição automática ocorre, garantindo uma condição segura para manutenção ou situações emergenciais. Após a desativação do modo, o sistema reinicia sua operação a partir do estado inicial configurado.

III. REQUISITOS FUNCIONAIS

Os requisitos funcionais do sistema foram definidos com o objetivo de estabelecer o comportamento esperado do controlador semaforico em diferentes condições de operação, incluindo o funcionamento normal do cruzamento, o controle de travessia de pedestres e os modos de emergência. Esses requisitos serviram como base para a modelagem formal do sistema no UPPAAL, permitindo a construção dos autômatos temporizados e a posterior etapa de verificação formal das propriedades especificadas. A Tabela I apresenta o conjunto completo de

requisitos funcionais considerados no desenvolvimento do modelo.

Tabela I: Requisitos Funcionais do Sistema

ID	Requisito Funcional
RF-01	O sistema deve controlar 3 semáforos independentes, sendo um associado à Rua A, outro associado à Rua B, outro associado ao sinalero dos pedestres.
RF-02	O sistema deve iniciar sua operação com todos os sinais em vermelho.
RF-03	O sistema deve alternar automaticamente o direito de passagem entre Rua A e Rua B.
RF-04	O sistema deve garantir que apenas uma rua possua sinal verde por vez.
RF-05	O sistema deve possuir para os sinais associados a rua A e B um estado amarelo antes da troca do semáforo para vermelho.
RF-06	O estado amarelo deve permanecer ativo durante um intervalo de tempo configurado.
RF-07	Após o estado amarelo, o sistema deve colocar ambos os semáforos em vermelho durante um intervalo de segurança.
RF-08	O sistema deve permitir que veículos provenientes da Rua A continuem na Rua A ou realizem conversão para a Rua B quando o semáforo da Rua A estiver verde.
RF-09	O sistema deve permitir que veículos provenientes da Rua B continuem na Rua B ou realizem conversão para a Rua A quando o semáforo da Rua B estiver verde.
RF-10	O sistema deve possuir um botão de solicitação de travessia de pedestres localizados nos cantos do cruzamento.
RF-11	Quando um botão de pedestre for pressionado, o sistema deve registrar a solicitação de travessia.
RF-12	O sistema deve atender solicitações de travessia somente após o encerramento seguro do ciclo atual de veículos.
RF-13	Durante a travessia de pedestres, ambos os semáforos de veículos devem permanecer em vermelho.
RF-14	O sistema deve liberar a travessia de pedestres durante um intervalo de tempo pré-definido.
RF-15	Após o término da travessia de pedestres, o sistema deve retornar automaticamente ao controle normal do tráfego.
RF-16	O sistema deve possuir um botão dedicado ao acionamento do modo emergência/manutenção.
RF-17	O sistema deve concluir a transição segura do estado atual antes de entrar em emergência.
RF-18	Durante o modo emergência/manutenção, todos os semáforos devem permanecer em vermelho.
RF-19	Durante o modo emergência/manutenção, nenhuma transição automática de estados deve ocorrer.
RF-20	Quando o modo emergência/manutenção for desativado, o sistema deve reiniciar sua operação a partir do estado inicial configurado.
RF-21	O sistema deve operar em execução cíclica infinita.
RF-22	O sistema deve utilizar temporizadores internos para controle das transições de estados.
RF-23	O sistema deve garantir que, após uma travessia de pedestres, pelo menos um ciclo de liberação da Rua A e um ciclo de liberação da Rua B ocorram antes que uma nova solicitação de travessia de pedestres seja atendida.

IV. SOFTWARE DO CONTROLADOR

O controlador foi implementado como uma máquina de estados finita temporizada, em linguagem C, em que toda a lógica do sistema está concentrada na função `trafficControllerTick()`. Essa função é chamada periodicamente e representa o avanço do tempo discreto do sistema. A cada chamada, um contador interno (`gStateTimer`) é incrementado e utilizado para decidir quando ocorre a transição entre estados.

Cada estado do sistema corresponde diretamente a uma configuração dos semáforos, e essa configuração é aplicada pela função `updateOutputs()`. Assim, a lógica de controle fica desacoplada da definição das saídas. Sempre que ocorre uma mudança de estado, a função `changeState()` é chamada, reiniciando o temporizador do estado e atualizando imediatamente os sinais de saída.

As transições são baseadas principalmente em tempo. Por exemplo, quando o sistema está em `STATE_A_GREEN`, ele permanece nesse estado até que o tempo definido para o verde da Rua A seja atingido, em seguida, transita para o amarelo, depois para um estado intermediário com todos os sinais em vermelho, e então libera a Rua B. Esse padrão se repete de forma cíclica. Durante esses pontos de transição segura (estados em que todos os semáforos estão vermelhos), o sistema avalia eventos pendentes, como solicitações de pedestres ou ativação de emergência.

As requisições externas são tratadas por meio de flags globais. Quando o botão de pedestre é pressionado, a variável `gPedestrianRequest` é ativada. O sistema não reage imediatamente a essa solicitação, em vez disso, ela é verificada apenas em estados seguros. Se não houver bloqueio ativo, o controlador entra no estado de pedestre, zera as flags de ciclo e impede novos atendimentos até que os ciclos de ambas as vias sejam completados novamente.

O modo de emergência segue a mesma lógica, ao pressionar o botão, a flag `gEmergencyPending` é ativada, mas a transição para o estado de emergência só ocorre quando o sistema atinge um estado seguro. Isso garante que nenhuma mudança abrupta ocorra durante um sinal verde ou amarelo. Uma vez em emergência, o sistema permanece estático (todos os sinais em vermelho) até que o modo seja explicitamente desativado.

De forma mais clara, a `main()` realiza uma simulação do controlador com duração de 100 ticks, em que cada tick representa uma unidade de tempo. Os semáforos A e B permanecem nos estados verde e amarelo por 10 ticks e 3 ticks, respectivamente. O intervalo em que ambos ficam em vermelho é de 2 ticks, enquanto o tempo de travessia para pedestres é de 8 ticks.

A simulação inicia no estado inicial e, em seguida, o controlador entra em operação normal. Nos instantes 15 e 20 ticks, o botão de solicitação de travessia de pedestres

é acionado. Nesse caso, espera-se apenas uma transição para o estado de travessia, pois o intervalo de 5 ticks entre os acionamentos não é suficiente para que ambas as ruas completem novamente seus ciclos.

No instante 40 ticks, o botão de emergência é pressionado e liberado apenas em 70 ticks. Durante esse período, espera-se que a máquina de estados interrompa as transições automáticas e permaneça continuamente no estado de emergência por 30 ticks. Após a liberação do botão, espera-se que o controlador retome as transições automáticas entre os estados, respeitando os tempos configurados para cada semáforo, até que a simulação seja encerrada em 100 ticks.

Os resultados podem ser obtidos executando o código do controlador, hospedado no repositório do GitHub disponível em https://github.com/JeanDapper/verificacao_formal_semaforo. Além disso, as demais informações do projeto, incluindo os testes unitários e a verificação formal, também podem ser encontradas no mesmo repositório.

V. TESTES UNITÁRIOS

Com o objetivo de validar o comportamento individual das funções do controlador, foram desenvolvidos testes unitários utilizando a biblioteca `unity` na linguagem C. Os testes foram elaborados a partir da definição de classes de equivalência para cada função implementada, permitindo verificar o comportamento esperado do sistema para diferentes condições de entrada.

Como exemplo representativo, esta seção apresenta as classes de equivalência, os casos de teste e a implementação dos testes unitários para uma das funções do controlador. As demais tabelas e implementações seguem a mesma estrutura adotada neste exemplo, e estão no anexo A do trabalho.

Ao total, foram implementados 44 casos de teste unitário, abrangendo todas as funções do controlador. Todos os testes executados foram aprovados sem falhas, indicando conformidade entre o comportamento esperado e a implementação das funções desenvolvidas.

A. Exemplo função `updateOutputs()`

A função `updateOutputs()`, é responsável por atualizar as saídas do sistema de semáforo com base no estado atual armazenado na variável global `gState`.

Ela implementa uma estrutura switch-case para verificar qual é o estado atual da máquina de estados e, a partir disso, definir as luzes dos semáforos de veículos e pedestres na estrutura global gLights.

A estrutura gLights possui três saídas principais:

- roadA: semáforo da via A;
- roadB: semáforo da via B;
- pedestrian: semáforo de pedestres.

Cada estado da máquina define uma configuração específica das luzes.

Além disso, o bloco default atua como mecanismo de segurança. Caso gState contenha um valor inválido ou inesperado, o sistema força todas as luzes para vermelho, evitando estados inseguros no cruzamento.

As classes definidas para esta função, vide Tabela II, seguem de forma geral, a regra de que cada caso deste tipo de estrutura de código deve ser tratado como uma classe independente, mas devido a implementação do código assumir que STATE_ALL_RED_TO_A e STATE_ALL_RED_TO_B possuem exatamente a mesma implementação, ambos foram considerados apenas uma classe, mas de qualquer maneira, os casos de teste considerados para esta classe consideram os dois valores possíveis da classe. Enquanto nas demais, foi utilizado apenas um valor de teste, conforme Tabela III.

Tabela II: Classes de equivalência da função updateOutputs

Classe	Descrição	Domínio da entrada
C1	Estado inicial / tudo vermelho	gState = 0
C2	Rua A verde	gState = 1
C3	Rua A amarelo	gState = 2
C4	Rua B verde	gState = 4
C5	Rua B amarelo	gState = 5
C6	Todos vermelho	gState = 3 ou 6
C7	Pedestre verde	gState = 7
C8	Emergência	gState = 8
C9	Estado inválido	gState < 0 ou gState > 8

Na implementação do controlador, não é possível acessar diretamente a variável gState para a realização dos testes. Dessa forma, foram utilizadas funções auxiliares para alterar indiretamente seu valor e, posteriormente, ve-

Tabela III: Casos de teste da função updateOutputs

Classe	Entrada (gState)	Saída Esperada
C1	0	A=RED, B=RED, PED=RED
C2	1	A=GREEN, B=RED, PED=RED
C3	2	A=YELLOW, B=RED, PED=RED
C4	4	A=RED, B=GREEN, PED=RED
C5	5	A=RED, B=YELLOW, PED=RED
C6	3	A=RED, B=RED, PED=RED
C6	6	A=RED, B=RED, PED=RED
C7	7	A=RED, B=RED, PED=GREEN
C8	8	A=RED, B=RED, PED=RED
C9	9	A=RED, B=RED, PED=RED

rificar se as saídas produzidas correspondem ao comportamento esperado. Para a função em questão, a máquina de estados do controlador foi executada pelo tempo exato necessário para que alcançasse o estado desejado para cada caso de teste, garantindo o valor desejado na variável gState. Isto foi feito através da função runTicks(). Para conferir o resultado, foram utilizadas as funções getRodaA(), getRoadB() e getPedestrian() que retornam qual cor do semáforo está ligada em cada rua. O código que implementa a função do caso de teste 1 da Tabela III pode ser visto no Código 1. O resultado do teste automático de todos os casos de teste implementados nesta função, pode ser visto na Tabela IV.

```

1 void test_updateOutputs_STATE_A_GREEN(void)
2 {
3     unsigned int targetTicks = TIME_ALL_RED;
4
5     runTicks(targetTicks);
6
7     TEST_ASSERT_EQUAL(
8         LIGHT_GREEN,
9         getRoadA()
10    );
11

```

```

12     TEST_ASSERT_EQUAL (
13         LIGHT_RED,
14         getRoadB ()
15     );
16
17     TEST_ASSERT_EQUAL (
18         PED_RED,
19         getPedestrian ()
20     );
21 }

```

Código 1: Teste do estado STATE_A_GREEN

Tabela IV: Resultados dos testes da função updateOutputs

#	Teste Executado	Resultado
1	test_updateOutputs_STATE_INIT	PASS
2	test_updateOutputs_STATE_A_GREEN	PASS
3	test_updateOutputs_STATE_A_YELLOW	PASS
4	test_updateOutputs_STATE_B_GREEN	PASS
5	test_updateOutputs_STATE_B_YELLOW	PASS
6	test_updateOutputs_ALL_RED_TO_B	PASS
7	test_updateOutputs_ALL_RED_TO_A	PASS
8	test_updateOutputs_PEDESTRIAN	PASS
9	test_updateOutputs_EMERGENCY	PASS
10	test_updateOutputs_INVALID	PASS
Total de testes		10
Falhas		0
Ignorados		0

VI. MODELAGEM DO SISTEMA

A modelagem do sistema foi realizada utilizando autômatos temporizados no UPPAAL, representando o comportamento do software do controlador por meio de máquinas de estados concorrentes sincronizadas através de canais do tipo broadcast. Para garantir maior fidelidade entre o modelo formal e a implementação real, todas as funcionalidades relevantes do código foram analisadas e transcritas para o modelo de simulação. Sempre que possível, foram mantidos os mesmos nomes de variáveis, estados e eventos utilizados no software original, com o objetivo de preservar a equivalência comportamental entre o controlador implementado e o modelo verificado formalmente.

O template Controlador, conforme Figura 2, representa a lógica principal do sistema. Nele foram modelados todos os estados operacionais do cruzamento, descritos a seguir:

- STATE_INIT: estado inicial do sistema, responsável por garantir que todos os semáforos iniciem em vermelho.
- STATE_A_GREEN: estado em que a Rua A possui sinal verde para circulação de veículos.
- STATE_A_YELLOW: estado de transição da Rua A, indicando a troca iminente do verde para vermelho.
- STATE_ALL_RED_TO_B: estado intermediário de segurança em que todos os semáforos permanecem em vermelho antes da liberação da Rua B.
- STATE_B_GREEN: estado em que a Rua B possui sinal verde para circulação de veículos.
- STATE_B_YELLOW: estado de transição da Rua B, indicando a troca iminente do verde para vermelho.
- STATE_ALL_RED_TO_A: estado intermediário de segurança em que todos os semáforos permanecem em vermelho antes da liberação da Rua A.
- STATE_PEDESTRIAN: estado responsável pela travessia segura de pedestres, mantendo ambos os fluxos veiculares bloqueados.
- STATE_EMERGENCY: estado especial de emergência ou manutenção, no qual todos os semáforos permanecem em vermelho.

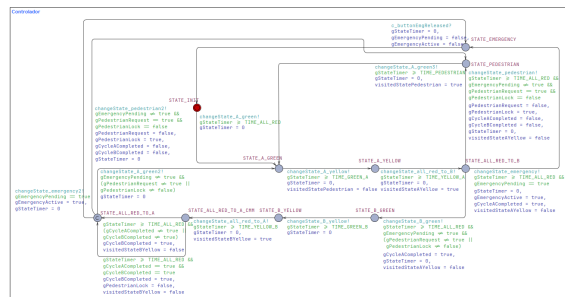


Figura 2: Máquina de estado, controlador

A comunicação entre os templates foi implementada utilizando canais do tipo broadcast chan, permitindo sincronização simultânea entre controlador e semáforos. Quando o controlador altera seu estado interno, um evento broadcast é emitido e recebido pelos semáforos correspondentes, atualizando seus estados físicos de forma determinística e sincronizada.

As transições entre estados foram controladas por relógios temporizados (clock) e guardas temporais. O relógio global gStateTimer foi utilizado para controlar o tempo de permanência em cada estado, permitindo

modelar os tempos mínimos de verde, amarelo, travessia de pedestres e intervalo de segurança “all red”. Dessa forma, cada mudança de estado ocorre somente após o tempo configurado ser atingido, representando o comportamento temporal real de um controlador semafórico.

Os componentes Semaforo_A, Semaforo_B e Semaforo_pedestre, vide Figura 3, foram modelados como máquinas de estados independentes contendo os estados físicos dos semáforos (verde, amarelo e vermelho). Esses componentes não tomam decisões próprias, sendo totalmente sincronizados pelo controlador através de canais broadcast, como `changeState_A_green`, `changeState_B_yellow` e `changeState_pedestrian`. Essa abordagem permitiu separar a lógica de controle da representação física dos atuadores.

O modelo foi desenvolvido buscando reproduzir a lógica implementada no software embarcado do controlador. Para isso, foram preservadas as mesmas regras de transição de estados existentes na implementação real, incluindo diferentes condições de operação, como ciclos normais, travessia de pedestres e modo de emergência. Como consequência, algumas transições entre estados utilizam múltiplos canais de sincronização distintos, mesmo quando possuem a mesma origem e destino de estados, permitindo representar os diferentes contextos operacionais previstos no software original.

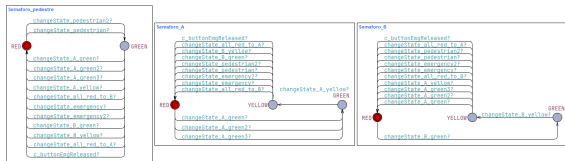


Figura 3: Máquina de estado, semaforos

Os botões de pedestre e emergência também foram modelados como autômatos independentes, conforme Figura 4. O template `Botao_pedestre` representa a geração de solicitações de travessia através da variável global `gPedestrianRequest`, enquanto o template `Botao_emergencia` representa a ativação e desativação do modo de emergência por meio das variáveis `gEmergencyPending` e `gEmergencyActive`. Essas variáveis globais permitem que o controlador reaja de forma assíncrona aos eventos externos.

Foram introduzidas variáveis auxiliares, que podem ser vistas na Figura 5, com o objetivo de representar

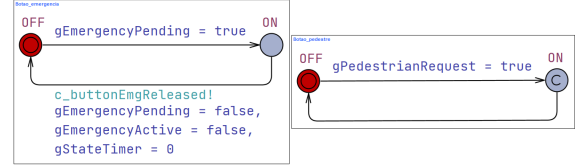


Figura 4: Máquina de estado, semaforos

propriedades comportamentais relevantes do sistema e possibilitar sua verificação formal. As variáveis `visitedStateAYellow` e `visitedStateBYellow` foram utilizadas para registrar a passagem prévia pelos estados amarelos das vias A e B, permitindo validar formalmente que toda transição do verde para o vermelho ocorre obrigatoriamente através do estado amarelo correspondente. De maneira semelhante, a variável `visitedStatePedestrian` foi empregada para indicar que a máquina de estados executou previamente o estado `STATE_PEDESTRIAN`, possibilitando a verificação de propriedades relacionadas ao ciclo de travessia de pedestres e ao retorno seguro do sistema ao funcionamento normal.

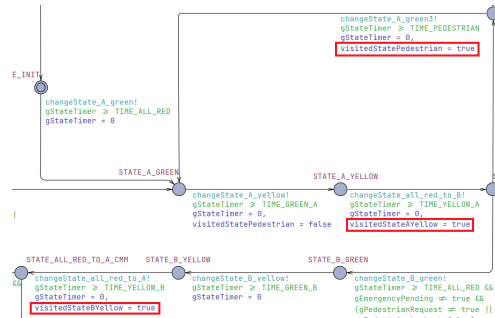


Figura 5: Máquina de estado, variaveis auxiliares

VII. VERIFICAÇÃO FORMAL DO MODELO

Após a modelagem, os requisitos funcionais do sistema foram formalizados em lógica temporal computacional (CTL) por meio das consultas do verificador do UPPAAL. Essas propriedades verificam aspectos de segurança, alcançabilidade, ausência de deadlock, sincronização correta dos semáforos, comportamento temporal e funcionamento seguro dos modos de emergência e travessia de pedestres.

- RF-01 - O sistema deve controlar 3 semáforos independentes, sendo um associado à Rua A, outro associado à Rua B, outro associado ao sinaleiro dos pedestres

```

E<> Semaforo_A.RED
E<> Semaforo_A.GREEN
E<> Semaforo_A.YELLOW
E<> Semaforo_B.RED
E<> Semaforo_B.GREEN
E<> Semaforo_B.YELLOW
E<> Semaforo_pedestre.RED
E<> Semaforo_pedestre.GREEN

```

- RF-02 - O sistema deve iniciar sua operação com todos os sinaleiros em vermelho

```

A[] ( Controlador.STATE_INIT imply
Semaforo_A.RED && Semaforo_B.RED
&& Semaforo_pedestre.RED )

```

- RF-03 - O sistema deve alternar automaticamente o direito de passagem entre Rua A e Rua B

```

E<> Controlador.STATE_A_GREEN
imply not Controlador.STATE_B_GREEN
E<> Controlador.STATE_B_GREEN
imply not Controlador.STATE_A_GREEN

```

- RF-04 - O sistema deve garantir que apenas uma rua possua sinal verde por vez

```

A[] not (Semaforo_A.GREEN &&
Semaforo_B.GREEN)
A[] not (Semaforo_A.GREEN
&& Semaforo_B.GREEN &&
Semaforo_pedestre.GREEN)

```

- RF-05 - O sistema deve possuir para os sinaleiros associados a rua A e B um estado amarelo antes da troca do semáforo para vermelho

```

A[] (Controlador.STATE_ALL_RED_TO_B
imply visitedStateAYellow == true)
A[] (Controlador.STATE_ALL_RED_TO_A_CMM
imply visitedStateBYellow == true)

```

- RF-06 - O estado amarelo deve permanecer ativo durante um intervalo de tempo configurado

```

A[] ( Controlador.STATE_A_YELLOW
&& gStateTimer <
TIME_YELLOW_A imply not
Controlador.STATE_ALL_RED_TO_B
)
A[] ( Controlador.STATE_B_YELLOW
and gStateTimer <
TIME_YELLOW_B imply not
Controlador.STATE_ALL_RED_TO_A_CMM
)

```

- RF-07 - Após o estado amarelo, o sistema deve colocar ambos os semáforos em vermelho durante um intervalo de segurança

```

A[] (Controlador.STATE_ALL_RED_TO_B
imply Semaforo_A.RED &&
Semaforo_B.RED)
A[] (Controlador.
STATE_ALL_RED_TO_A_CMM imply
Semaforo_A.RED && Semaforo_B.RED)

```

- RF-10 - O sistema deve possuir um botão de solicitação de travessia de pedestres localizados nos cantos do cruzamento

```

E<> Botao_pedestre.OFF
E<> Botao_pedestre.ON

```

- RF-11 - Quando um botão de pedestre for pressionado, o sistema deve registrar a solicitação de travessia

```

A[] ( Botao_pedestre.ON imply
gPedestrianRequest == true )

```

- RF-12 - O sistema deve atender solicitações de travessia somente após o encerramento seguro do ciclo atual de veículos

```

A[] (Semaforo_pedestre.GREEN
imply (Semaforo_A.RED &&
Semaforo_B.RED))

```

- RF-13 - Durante travessia ambos vermelhos - Durante a travessia de pedestres, ambos os semáforos de veículos devem permanecer em vermelho

```

A[] (Semaforo_pedestre.GREEN
imply (Semaforo_A.RED &&
Semaforo_B.RED))

```

- RF-15 - Após o término da travessia de pedestres, o sistema deve retornar automaticamente ao controle normal do tráfego

```

A[] ((Controlador.STATE_A_GREEN
visitedStatePedestrian == true)
imply visitedStatePedestrian ==
true)

```

- RF-16 - O sistema deve possuir um botão dedicado ao acionamento do modo emergência/manutenção

```

E<> Botao_emergencia.OFF
E<> Botao_emergencia.ON

```

- RF-17 - O sistema deve concluir a transição segura do estado atual antes de entrar em emergência

```

A[] (gEmergencyActive imply
Controlador.STATE_EMERGENCY)

```


- RF-18 - Durante o modo emergência/manutenção, todos os semáforos devem permanecer em vermelho

```
A[] (Controlador.STATE_EMERGENCY
  imply (Semaforo_A.RED
    && Semaforo_B.RED &&
    Semaforo_pedestre.RED))
```

- RF-19 - Durante o modo emergência/manutenção, nenhuma transição automática de estados deve ocorrer

```
A[] (Controlador.STATE_EMERGENCY
  imply not Controlador.STATE_A_GREEN)
```

- RF-20 - Quando o modo emergência/manutenção for desativado, o sistema deve reiniciar sua operação a partir do estado inicial configurado.

```
(not gEmergencyActive &&
  Controlador.STATE_EMERGENCY) ->
  Controlador.STATE_INIT
```

- RF-21 - O sistema deve operar em execução cíclica infinita

```
A[] not deadlock
```

- RF-23 - O sistema deve garantir que, após uma travessia de pedestres, pelo menos um ciclo de liberação da Rua A e um ciclo de liberação da Rua B ocorram antes que uma nova solicitação de travessia de pedestres seja atendida

```
A[] Controlador.STATE_PEDESTRIAN
  imply (gCycleACompleted == true &&
  gCycleBCompleted == true)
```

```
/* RF-01A */ Eo Semaforo_A.RED
/* RF-01B */ Eo Semaforo_A.GREEN
/* RF-01C */ Eo Semaforo_A.YELLOW
/* RF-01D */ Eo Semaforo_B.RED
/* RF-01E */ Eo Semaforo_B.GREEN
/* RF-01F */ Eo Semaforo_B.YELLOW
/* RF-01G */ Eo Semaforo_pedestre.RED
/* RF-01H */ Eo Semaforo_pedestre.GREEN
/* RF-02 */ A[] (Controlador.STATE_INIT imply Semaforo_A.RED && Semaforo_B.RED && Semaforo_pedestre.RED)
/* RF-03A */ Eo Controlador.STATE_A_GREEN imply not Controlador.STATE_B_GREEN
/* RF-03B */ Eo Controlador.STATE_B_GREEN imply not Controlador.STATE_A_GREEN
/* RF-04A */ A[] not (Semaforo_A.GREEN && Semaforo_B.GREEN)
/* RF-04B */ A[] not (Semaforo_A.GREEN && Semaforo_B.GREEN && Semaforo_pedestre.GREEN)
/* RF-05A */ A[] (Controlador.STATE_ALL_RED_TO_B imply visitedStateYellow == true)
/* RF-05B */ A[] (Controlador.STATE_ALL_RED_TO_A_CW imply visitedStateYellow == true)
/* RF-06A */ A[] (Controlador.STATE_A_YELLOW and gStateTimer < TIME_YELLOW_A imply not Controlador.STATE_ALL_RED_TO_B)
/* RF-06B */ A[] (Controlador.STATE_B_YELLOW and gStateTimer < TIME_YELLOW_B imply not Controlador.STATE_ALL_RED_TO_A_CW)
/* RF-07A */ A[] (Controlador.STATE_ALL_RED_TO_B imply Semaforo_A.RED && Semaforo_B.RED)
/* RF-07B */ A[] (Controlador.STATE_ALL_RED_TO_A_CW imply Semaforo_A.RED && Semaforo_B.RED)
/* RF-10A */ Eo Botao_pedestre.OFF
/* RF-10B */ Eo Botao_pedestre.ON
/* RF-11 */ A[] ( Botao_pedestre.ON imply gPedestrianRequest == true )
/* RF-12 */ A[] (Semaforo_pedestre.GREEN imply (Semaforo_A.RED && Semaforo_B.RED))
/* RF-13 */ A[] (Semaforo_pedestre.GREEN imply (Semaforo_A.RED && Semaforo_B.RED))
/* RF-15 */ A[] ((Controlador.STATE_A_GREEN && visitedStatePedestrian == true) imply visitedStatePedestrian == true)
/* RF-16A */ Eo Botao_emergencia.OFF
/* RF-16B */ Eo Botao_emergencia.ON
/* RF-17 */ A[] (gEmergencyActive imply Controlador.STATE_EMERGENCY)
/* RF-18 */ A[] (Controlador.STATE_EMERGENCY imply (Semaforo_A.RED && Semaforo_B.RED && Semaforo_pedestre.RED))
/* RF-19 */ A[] (Controlador.STATE_EMERGENCY imply not Controlador.STATE_A_GREEN)
/* RF-20 */ A[] (not gEmergencyActive && Controlador.STATE_EMERGENCY -> Controlador.STATE_INIT)
/* RF-21 */ A[] not deadlock
/* RF-23 */ A[] Controlador.STATE_PEDESTRIAN imply (gCycleACompleted == true && gCycleBCompleted == true)
```

Figura 6: Verificação formal das propriedades

Após a etapa de verificação formal, observou-se que todos os requisitos especificados foram satisfeitos, com exceção do requisito RF-23, conforme Figura 6. A violação desse requisito ocorre devido à lógica atualmente

implementada no software, na qual as variáveis responsáveis pela indicação de conclusão dos ciclos das ruas A e B (gCycleACompleted e gCycleBCompleted) não garantem adequadamente que ambos os fluxos veiculares completem seus ciclos antes que uma nova solicitação de travessia de pedestres seja atendida. Como consequência, existe ao menos um caminho de execução no modelo em que o estado de travessia de pedestres pode ser alcançado sem que a condição de fairness definida pelo requisito seja plenamente satisfeita.

Mais especificamente, foi identificado um cenário em que o estado STATE_B_GREEN nunca é alcançado. Isso ocorre quando o pedestre aciona repetidamente o botão de travessia enquanto o sistema se encontra nos estados STATE_A_GREEN ou STATE_A_YELLOW. Nesse caso, após a Rua A concluir seu ciclo e entrar no estado de todos os sinais vermelhos, o controlador direciona o sistema para o estado STATE_PEDESTRIAN. Ao término da travessia, o sistema retorna novamente ao estado STATE_A_GREEN, permitindo que uma nova solicitação de pedestre reinicie o mesmo comportamento. Dessa forma, o ciclo da Rua B pode ser indefinidamente adiado, caracterizando a violação do requisito de fairness estabelecido no RF-23.

VIII. CONCLUSÃO

Este trabalho apresentou o desenvolvimento, teste e verificação formal de um controlador semafórico embarcado em tempo real. Os testes unitários realizados com Unity validaram corretamente as funções implementadas, com todos os casos de teste aprovados. Entretanto, a verificação formal no UPPAAL identificou um problema de fairness não detectado pelos testes tradicionais, no qual o sistema pode entrar indefinidamente no ciclo “Rua A → Pedestre → Rua A → Pedestre”, impedindo a liberação da Rua B. Dessa forma, foi possível observar que testes unitários e verificação formal são técnicas complementares, já que os testes validam comportamentos específicos enquanto a verificação formal permite analisar todos os caminhos possíveis de execução do sistema, aumentando sua confiabilidade e segurança.